Aristotle University of Thessaloniki

Faculty of Sciences - School of Informatics

Regev's Algorithm

Author:Papadopoulos Georgios

Supervisor:Konofaos Nikolaos

A thesis submitted in fulfillment of the requirements for the degree of Bachelor

March 2026

Declaration

I hereby declare that except where specific reference is made to the work of others, the contents of this dissertation are original and have not been submitted in whole or in part for consideration for any other degree or qualification in this, or any other university. This dissertation is my own work and contains nothing which is the outcome of work done in collaboration with others, except as specified in the text and acknowledgements.

"Mathematics is the art of giving the same name to different things." .

— Henri Poincaré

"Not only does God play dice but... he sometimes throws them where they cannot be seen."

— Steven Hawking

Abstract

The purpose of this thesis is to analyze the mathematical formulation behind Regev's quantum algorithm, explain its significance, practical functionality, and limitations, and demonstrate a way in which it could be implemented.

Specifically, we will begin by briefly explaining the idea behind Shor's algorithm, describing the classical and quantum subroutines, and analyzing its complexity. Following this, we will discuss certain mathematical concepts that provide the foundation for the generalizations and improvements introduced by Regev, such as the connection between finding periods in 1 dimension and determining bases for N-dimensional lattices,the probability of finding quadratic non-residues modulo N, the Quantum Fourier Transform and its relation to the dual lattice and the Lenstra–Lenstra–Lovász algorithm.

Next, we will comprehensively explain Regev's algorithm and its theoretical complexity with respect to both gate and qubit requirements, and demonstrate a potential implementation in Qiskit. Finally, we describe certain improvements that have been suggested following the publication of the original paper.

Acknowledgments

I want to express my sincerest gratitude to my supervisor, Mr. Konofaos. Their insight and support were essential to both the direction and completion of this work.

Contents

Part I

Shor's Algorithm

Chapter 1

Introduction

Out of all the monumental achievements in quantum computing and the different algorithms that have been designed throughout the years, Shor's algorithm perhaps remains as the greatest demonstration of what quantum computers are capable of. Often designated as the crown jewel of the field, it is the first algorithm that truly presents an idea that manages to affect our everyday lives. Its mere existence,along with the possibilities that said existence implies,has even inspired the development of new fields like post-quantum cryptography, that aims to create new challenges upon which modern encryption can be based. All of that purely based on the fact that it completely redefines one of oldest problems in number theory: prime factorization

Chapter 2

The idea

First, it is important to define the objective. Given a (usually large) composite number N whose factors are p and q (meaning $N = p \cdot q$), the task is to find the numbers p and q. This can in turn be extended to find the full prime factorization of N by recursively applying the algorithm to p and q. Hence, we will only be dealing with the case in which p and q are prime numbers. From now and throughout the paper, we define $n = \lceil log_2 N \rceil$ as the number of bits in N.

Furthermore, there are certain trivial cases for N. For example, we can easily check whether N is even in $O(logN)$ using Euclid's algorithm, or a prime power ($N = p^k$ for some p), with other classical algorithms. This means that we only need to look at the case where N is an odd composite number that is not a prime power.

The classical part of Shor's algorithm begins by picking a random number a with $2 < a < N$, and running Euclid's algorithm to compute $gcd(a, N)$. If $\gcd(a, N) > 1$, then we have a non trivial factor,and the other factor can be found by simply dividing $N / \gcd(a, N)$. Otherwise,it follows that a and N are coprime and a is an element of the multiplicative group of the integers modulo N, meaning it has a multiplicative inverse modulo N. This also implies:

$$a^r \equiv 1 \pmod{N} \tag{2.1}$$

for some r with $1 < r < \Phi(N)$[1], where r is called the multiplicative order, or **period**, of a modulo N and $\Phi(N)$ is Euler's totient function. It should be noted that r is defined as the smallest possible number that satisfies (2.1).

In this case, Shor's algorithm applies a quantum subroutine to find r. We will analyze this subroutine in the following chapter, as it directly ties into Regev's algorithm.

From (2.1), it is obvious that $N \mid (a^r - 1)$. This implies (but is not equivalent to):

$$N \mid (a^{r/2} - 1)(a^{r/2} + 1) \tag{2.2}$$

From (2.2), a few things can be seen. First, our logic from now on will work only for even r. In the case that r happens to be odd, Shor picks a new a, computes a new r,and relies on probabilistically finding an even r.

Also, it cannot be the case that $N \mid (a^{r/2} - 1)$, since that would imply that $a^{r/2} \equiv 1 \pmod{N}$, which contradicts the assumption that r is the smallest possible number that satisfies (2.1). At this point,we have two possibilities: $N \mid (a^{r/2} + 1) \implies a^{r/2} \equiv -1 \pmod{N}$,in which case $\gcd(a^{r/2} - 1, N) = 1$, and we do not have any nontrivial factors, or $N \nmid (a^{r/2} + 1) \implies a^{r/2} \not\equiv -1 \pmod{N}$,and therefore $d = gcd(a^{r/2} - 1, N)$ is a nontrivial prime factor of N,with the other being N/d, and the algorithm is finished.

To summarize the steps we just described:
1.Pick a number a between 2 and N
2.If $gcd(a, N) = 1$,then return a and N/a as nontrivial factors, else run a

[1] r is bounded up by $\Phi(N)$ due to Lagrange's theorem

quantum subroutine to find the order r of a modulo N

3.If r is even, return to step 1, else compute $d = \gcd(a^{r/2} - 1, N)$

4.If $d > 1$, return d and N/d, otherwise return to step 1.

Chapter 3

Quantum subroutine

The quantum part of Shor's algorithm addresses the following subproblem: Given a and N where $a < N$, $\gcd(a, N) = 1$, and N is an odd number that is not a prime power, find the multiplicative order of a modulo N. To solve this problem, Shor uses a variant of the quantum phase estimation algorithm which will be described extensively below.

First,we need to initialize 2 quantum registers.

1.The *exponent register*, with size n qubits.[1]
2.The *function register*, with size n qubits

First,we apply Hadamard gates to every qubit. This achieves the following superposition

$$\frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle |0\rangle \tag{3.1}$$

Afterwards,we perform modular exponentiation on the superposition and store the result in the function register. The following quantum operation

[1]In practice, it is optimal for the QFT to use a superposition over some number Q with $Q \approx N^2$,and an exponent register of $2n$. Otherwise, it is possible that several QFT peaks map to the same ratio in the continuous fraction, which increases the number of attempts required to determine r. However, optimizations have been made - see Chapter 4 for complexity.

is therefore performed on the function register:

$$|x\rangle\,|0\rangle \mapsto |x\rangle\,|a^x \mod N\rangle \tag{3.2}$$

Note that this means that each exponent register qubit $|x\rangle$ is entangled with a function register qubit $|a^x \mod N\rangle$. This means that the period information is now encoded in the exponent registers. Also note that some exponent registers will contain the same values. Specifically, for every register x_i and for each possible value $y_i = a^{x_i} \pmod N$, if we assume that x_0 is the smallest number that satisfies that equation for a given y_0, then it is also satisfied by $x_0 + r, x_0 + 2r \ldots$, up to a total of M numbers (including x_0), where $M = \lfloor (N - x_0 - 1)/r \rfloor$. Therefore, the exponent registers are in the following superposition:

$$|\psi\rangle = \frac{1}{\sqrt{M}} \sum_{x=0}^{M-1} |x_0 + kr\rangle \tag{3.3}$$

Now, the QFT on N elements is defined as the following operation:

$$|x\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} e^{2\pi i x y/N}\,|y\rangle \tag{3.4}$$

The next step is to apply QFT on the exponent registers in state ψ, giving the following state:

$$|\psi'\rangle = \frac{1}{\sqrt{MN}} \sum_{y=0}^{N-1} \Big(\sum_{k=0}^{M-1} e^{2\pi i k r y/N} \Big) e^{2\pi i x_0 y/N}\,|y\rangle \tag{3.5}$$

The amplitude of y is a geometric series. If we substitute the equation for the geometric sum of the inner series,we find that the sum is maximal when $1 - e^{2\pi i r y/N} \approx 0 \implies e^{2\pi i r y/N} \approx 1 \implies y \approx jN/r$ for $j = 0, 1 \ldots r - 1$[2]. Therefore, the measurement is more likely to measure a multiple of N/r, from which then r is extracted using continuous fractions.

[2]This should be a fairly intuitive explanation for the 1-dimensional case. The n-dimensional case is explored and analyzed further in the QFT chapter.

Chapter 4

Shor's complexity

The complexity of the quantum subroutine is the bottleneck of the algorithm. Gidney and Ekera [4] applied a variant of Shor's algorithm[1]that requires $O(n)$ qubits for the exponent register.

Now,we have $O(n)$ qubits for the exponent register, another $O(n)$ for the function register [17], and $O(n)$ ancilla qubits for the modular exponentiation,addition and multiplication [16]. That gives us a total of $O(3n) = O(n)$ qubits for space complexity. The best optimized version, Beauregard's construction, uses $2n + 3$ qubits.

In terms of gate complexity, we need to perform approximately $O(n)$ multiplications that can be performed in $O(n^2)$ gates [14] using optimized schoolbook multiplication, for a total of $O(n^3)$ gates. QFT with approximations requires $O(nlogn)$ gates [9],and the continued fractions step requires approximately $O(n^2)$ classical bit operations[1]. The dominant term is, of course, $O(n^3)$

Finally, recent analysis [3] has proven that once the order of a randomly chosen element of $(\mathbb{Z}/N\mathbb{Z})^*$ has been obtained, regardless of whether the

[1]The original algorithm is due to Ekera and Hastad from 2017, see citation [25] in the paper cited

order is odd or even, N can be recovered with very high probability[2] using classical post-processing. Odd orders affect the runtime,but the asymptotic complexity remains the same.

In conclusion,we have:
Gate complexity: $O(n^3)$
Space complexity: $O(n)$
Independent runs: 1

[2]The only points of concern are trivial values for r, but the probability of finding one of these tends to 0 as N tends to infinity, so for large values this probability is extremely low.

Part II

Mathematical Background

Chapter 5

Periods and Lattices

The first, and arguably most important, mathematical foundation for this paper would be understanding the abstract concept of a lattice, and how the shortest vector in a lattice inherently quantifies the notion of period.

Definition 5.1 (Lattice) A lattice is a discrete additive subgroup of $\mathbb{R}^n$, i.e. it is a subset $\Lambda \subseteq \mathbb{R}^n$ satisfying the following properties:

1. Λ is closed under addition and subtraction.
2. $\exists\, \epsilon > 0$ such that for any two distinct lattice points $x \neq y \in \Lambda$, their distance is at least $\|x - y\| \geq \epsilon$.

In practice, this implies that a n-dimensional lattice Λ^n is the set L of all integer linear combinations of n linearly independent vectors. In other words:

$$\Lambda^n = \{\, z_1 b_1 + z_2 b_2 + \cdots + z_n b_n \mid z_i \in \mathbb{Z}, b_i \in \mathbb{R}^n \,\} \tag{5.1}$$

The vectors b_i form a basis of the lattice. The most common example of a lattice is $\mathbb{Z}^n$, the set of all n dimensional vectors with integer entries.

Now, given a lattice Λ, the **Shortest Vector Problem** (SVP) asks to find the non-zero vector with minimal Euclidean length. That is, we are looking for v_0, such that:

$$\|v_0\| \leq \|v\| \; \forall\, v \in \Lambda \tag{5.2}$$

This problem is, in general, NP-Hard[7].

Now, the Period Finding problem (PF) can be stated as follows:
Given a function $f : \mathbb{Z} \to S$ that is **periodic**, meaning that there exists some $r > 0$ s.t. $f(x + r) = f(x) \; \forall \; x$, find the smallest possible r that satisfies this equation.

We will now show that, in 1 dimension, SVP reduces to period finding.

Lemma 5.1 ($SVP \leq_P PF$**)** Define the set of periods P as follows:

$$P = \{ \, k \in \mathbb{Z} \mid f(x + k) = f(x) \; \forall \; x \, \} \tag{5.3}$$

Proof: For $a, b \in P$,we have $f(x + a) = f(x)$, $f(x + b) = f(x)$. Therefore, $f(x + (a + b)) = f((x + a) + b) = f(x + a) = f(x)$, so P is closed under addition.
Furthermore,$f(x - a) = f((x - a) + a) = f(x)$, which means that if $a \in P$,then $-a \in P$, so every element has an inverse. Hence,since P is closed under addition, it is also closed under subtraction. That proves the first lattice property for P.
Since r is by definition the smallest element of P, setting $\epsilon = r$ satisfies property 2, and therefore P is a lattice.

Moreover, since P is a subset of $\mathbb{Z}$ that is closed under addition and every element has an inverse, it follows that P is a subgroup of $\mathbb{Z}$. Every subgroup of $\mathbb{Z}$ is of the form $m\mathbb{Z} = \{ \, km : k \in \mathbb{Z} \, \}$,where m is the smallest element of the subgroup, so in this case $m = r$. Therefore, the elements of the lattice are $\ldots, -2r, -r, 0, r, 2r, \ldots$, and so the shortest nonzero elements are r and $-r$, meaning the magnitude of the shortest vector in this case is exactly r. This concludes our proof.

This can easily be generalized to n dimensions. Suppose that we have a function $f : \mathbb{Z}^n \to R$, which is periodic with respect to some arbitrary n-dimensional vector v. As stated above, a lattice can be constructed that

consists of all other possible vectors with respect to which f is periodic. However, now we are no longer looking to find the shortest vector according to Euclidean distance, since that would simply be a vector with 0 everywhere besides the dimension with the shortest period. Instead,we want to find the shortest period in **every** dimension, so the shortest **basis** with respect to Euclidean distance. Therefore, the problem of finding periods in n dimensions broadly corresponds to finding the shortest basis of a given n-dimensional lattice.

To explain the meaning of the word "broadly" there, we first need to define what a coset is

Definition 5.2 (Coset) Given some group G and some subgroup $H \subseteq G$, a coset with representative g for some given $g \in G$ is simply the set H_g defined as $H_g = \{ g + h \,|h \in H \}$.

We can think of H_g as being essentially H shifted by g.

In reality, both SVP and factoring are special cases of a more general problem, the **Hidden Subgroup Problem** (or HSP for short) for specific types of groups [1]. Regev himself proved in [12] that a quantum computer can solve SVP to some approximation factor (that is, find a vector whose magnitude is larger than the shortest to a constant factor) under the assumption that there exists an algorithm that solves the hidden subgroup problem on the dihedral group by coset sampling . Factoring corresponds to solving HSP on finite abelian groups. In fact, the classical postprocessing in Regev's algorithm (most of what we will describe in Chapters 9 and 10) is essentially already outlined in the paper above - Regev's new idea is to create a relationship that intertwines factoring and SVP, solve SVP with coset sampling, and use his solution to solve factoring as well.

[1] The reader can inform himself/herself more about HSP in the wikipedia page

Next, it is important to prove the following statement:

Lemma 5.2 The set of vectors V with entries $v_i \in \{ 0, 1 \dots N-1 \}$ such that for a fixed set of bases $b_1, b_2 \dots b_d$ it holds that

$$\prod_{i=1}^{d} b_i^{v_i} = 1 \pmod{N} \tag{5.4}$$

forms a subgroup of $\mathbb{Z}^d$ under vector addition, and therefore a lattice.[2]

Proof: Clearly, $V \subseteq \mathbb{Z}^d$. Consider the requirements for V to be a group:

Closure: $\prod_{i=1}^{d} b_i^{v_i+w_i} \pmod{N} = \prod_{i=1}^{d} b_i^{v_i} \cdot \prod_{i=1}^{d} b_i^{w_i} = 1 \cdot 1 = 1 \pmod{N}$, hence if $b, w \in V$ then $b + w \in V$

Inverse element: $\prod_{i=1}^{d} b_i^{-v_i} = 1 \pmod{N}$ since it is the modular inverse of the original product, which is congruent to 1 modulo N.

Identity element: Setting $\mathbf{v} = \mathbf{0}$ satisfies the relationship.

Associativity: Inherited by the parent group.

Therefore, the set of all vectors whose coordinates satisfy (5.4) forms a finite lattice Λ, and our proof is complete.

Consider that a finite n-dimensional lattice modulo N whose elements are vectors that satisfy the congruence in (5.4) is a subgroup of the group of all vectors with integer coordinates mod N (denoted Z_N^n), so we can define a coset of that lattice with respect to that group accordingly. Crucially, replacing 1 with u for some arbitrary $u \in \mathbb{Z}_N^n$ defines a coset whose representative is some exponent vector that represents the exact shift that must be applied to the v_i to get u instead of 1. This will become important later.

[2](5.4) is a linear congruence in disguise. To see this, consider what happens when taking the logarithm of both sides.

Lastly, in our next chapters we will be discussing a lot the notion of a dual lattice. Let us define it here:

Definition 5.3 (Dual lattice) The dual lattice Λ^* of a given lattice Λ, as follows:

$$\Lambda^* = \{ \mathbf{y} \in \mathbb{R}^n \mid \langle \mathbf{y}, \mathbf{x} \rangle \in \mathbb{Z} \quad \forall\, x \in \Lambda \} \tag{5.5}$$

where $\langle \mathbf{y}, \mathbf{x} \rangle$ denotes the inner product operation between the vectors $\mathbf{y}$ and $\mathbf{x}$.

Chapter 6

Regev's assumption from number theory

Regev's algorithm and its speedup crucially relies on a specific number theoretic assumption regarding products of prime numbers modulo N. Our overview could be considered complete without even remotely approaching this specific detail. Despite that, we chose to bore the reader with more abstract mathematics in the hopes that he stops reading the paper before he reads anything of real significance.

Now, let $b_1, b_2, \ldots, b_d$, be the first d primes in ascending order. Let X be defined as follows

$$X \equiv b_1^{z_1} b_2^{z_2} \ldots b_d^{z_d} \pmod{N} = \prod_{i=1}^{d} b_i^{z_i} \pmod{N} \tag{6.1}$$

for some $z_i \in \mathbb{Z}$. Suppose $\mathbf{z} = (z_1, z_2, \ldots, z_d)$ is a vector in $\mathbb{Z}^d$. The set of all vectors $\mathbf{z}$ that satisfy the modular multiplicative congruence $X^2 \equiv 1 \pmod{N}$ forms a lattice which we will call Λ. Furthermore, a sublattice of Λ can be constructed by the set of all vectors $\mathbf{z}$ that satisfy the congruence $X = \pm 1 \pmod{n}$. We will call this sublattice Λ_0, and denote by Λ / Λ_0 the set of vectors that are in Λ but not in Λ_0.

It is easy to see that finding a vector in Λ / Λ_0 is equivalent to finding a non-trivial factor of N. This is because N will then divide the product $(X-1)(X+1)$ but will not divide either of the terms, and therefore, as

explained in Chapter 1, computing $\gcd(X-1, N)$ gives us the nontrivial factor. A consequence of the Chinese Remainder Theorem is that for a number with k distinct prime factors, there are 2^k possible solutions to the congruence for Λ. In our case, this means that there are 4 possible solutions. The congruence for Λ_0 will obviously have only 2 solutions regardless of k. This means that half of our solutions are trivial! To avoid trivial solutions, Regev's algorithm requires the existence of a vector $\mathbf{z}$ where [1].

$$\|\mathbf{z}\| \leq 2^{O(d)} \tag{6.2}$$

However,at the time the original paper was published, the only way to guarantee that a sufficiently small vector would exist is a result found in the following paper [8].

Lemma 6.1: A vector satisfying (6.2) can be guaranteed to exist if the following upper bound holds

$$n(p) < (logp)^2 \tag{6.3}$$

where $n(p)$ denotes the least quadratic non-residue modulo p (p and q being the factors of N).

The above result, however, is dependent on the unproven Generalized Riemann Hypothesis [2], and hence the algorithm isn't really proven correct in the mathematical sense but rather heuristically conjectured to be working. Regev is satisfied with this heuristic result most likely due to the fact that most mathematicians regard GRH to be true. Let us briefly explain the connection between quadratic non-residues and non-trivial factors.

A **quadratic residue** (or QR) modulo N is defined as some number y that is congruent to a perfect square modulo N, i.e. there exists x such that

[1] For an explanation regarding this bound, go to the end of Chapter 9

[2] In fact, it is an expansion of an old classical result that states $n(p) < C(logp)^2$

$x^2 \equiv y \pmod{N}$. By the Chinese Remainder Theorem, this is equivalent to y being a QR modulo p **and** modulo q. A number that is not a QR is called a QNR, or Quadratic Non-Residue. Keeping the earlier notation $n(p)$ for the least QNR, let us state some facts for QRs and QNRs in our problem:

Fact 1. $n(p)$ is always prime, regardless of whether p is prime or not.[3]
Fact 2. If y is a QNR mod p, then it is also a QNR mod N.
Fact 3. If all b_i are QRs, then X is also a QR.

By factoring the congruence $X^2 \equiv 1 \pmod{N}$ as $(X-1)(X+1) \equiv 0 \pmod{N}$ we can see that for any $N = p \cdot q$, there are exactly 4 cases that produce a vector in Λ. These are as follows.

1. $X \equiv 1 \pmod{p}$ and $X \equiv 1 \pmod{q}$ (by CRT,$X \equiv 1 \pmod{N}$)
2. $X \equiv -1 \pmod{p}$ and $X \equiv -1 \pmod{q}$ (by CRT,$X \equiv -1 \pmod{N}$)
3. $X \equiv -1 \pmod{p}$ and $X \equiv 1 \pmod{q}$
4. $X \equiv 1 \pmod{p}$ and $X \equiv -1 \pmod{q}$

Consider the equation

$$X^2 \equiv 1 \pmod{N} \iff (X-1)(X+1) = k \cdot N = k \cdot p \cdot q$$

for some $k \in \mathbb{Z}$. In case 1, $X-1$ is some multiple of N and therefore $\gcd(X-1, N) = N$. This means that the only possible cases that produce a nontrivial result need X to be -1 modulo at least one of p and q. [4]

However, if $p \equiv 3 \pmod{4}$ and $q \equiv 3 \pmod{4}$ and X is a QR, then X cannot be -1 modulo either prime. This is because, under these conditions,

[3] If $n(p) = ab$ with $1 < a, b < n(p)$, then a and b have to be QRs, but then $n(p)$ is also a QR, so it cannot be the least QNR

[4] Case 2 also does not produce a nontrivial result. To see this, remember that the gcd algorithm would involve computing $\gcd(X-1 \pmod{N}, N)$. For $X \equiv -1 \pmod{N}$, this is the GCD of -2 and N, which is 1 because we initially assumed that N is odd.

-1 is a QNR modulo both p and q [5]. Then the only possible case is case 1, in which case the algorithm will fail.
Since we cannot know the congruence class of p and q beforehand, we need to guarantee that X is a QNR. Recall that according to fact 1, the least QNR is prime. Therefore, if all our primes are $< n(p)$, they have to all be QRs modulo p (and hence also QRs modulo N, according to fact 2), which makes X a QR and thus the algorithm will fail given $p \equiv 3 \pmod 4$ and $q \equiv 3 \pmod 4$.

This means that assuming GRH, combining the bound from (6.2) and the inequality $p \leq \sqrt{N}$, we get

$$n(p) < \frac{(logN)^2}{4}$$

In a followup paper, Cedric Pilatte[10], using a technique based on zero-density estimates for Dirichlet characters modulo N (the mathematics are beyond the scope of this thesis), proved the following result:

Lemma 6.2 (Pilatte): Given $b_1, b_2, \ldots, b_d$ independent and identically distributed random variables, each uniformly distributed in the set of primes $\leq d^{10^3 d}$ where $d = \lceil \sqrt{logN} \rceil$, every $x \in \langle b_1, b_2, ..., b_d \rangle$ (that is, the span of the b_i as a vector) can be expressed with high probability as

$$x \equiv \prod_{i=1}^{d} b_i^{z_i} \tag{6.4}$$

for some integers z_i with $|z_i| \leq 2^{O(d)}$ for all $1 \leq i \leq d$.

However, this places a very large upper bound on the primes, which severely affects gate complexity for any practical applications. More results need to be produced to guarantee practical functionality.

[5] This proof is based on Legendre symbols, which we encourage the reader to research on their own.

From what we said, it is implied that generally, if X is a QR, the algorithm fails. The reader might therefore wonder regarding the probability of the algorithm's success. As we established, half of the possible solutions are trivial. However, as we will find out later, if the algorithm is more likely to produce shorter exponent vectors -hence the bound $\|\mathbf{z}\| \leq 2^{O(d)}$ from earlier, it is also more likely to produce non-trivial solutions. Lastly, in regards to producing non-trivial solutions, let us state the following lemma:

Lemma 6.3 (Pomerance's theorem): Suppose G is a finite abelian group with minimal number of generators r. Then, when choosing elements from G independently and uniformly, the *expected* number of elements needed to generate G is less than $r + \sigma$, where $\sigma \approx 2.118$.

This implies that $r + 4$ elements are enough to generate G with probability $1/2$. This means that we necessarily need to have $\geq r + 4$ samples. We will use this theorem in the LLL chapter in order to improve the odds.

Chapter 7

Multiplication of small numbers

Regev presents us further with a potential way we could compute the product

$$X = \prod_{i=1}^{d} b_i^{e_i} \pmod{N}$$

efficiently. To do this, he combines two ideas that come from fast multiplication algorithms.

The first idea is the, by now well known, modular exponentiation algorithm for classical computers. Suppose we need to compute some large power k of a small number x modulo N where $N > 1$, so $x^k \pmod{N}$. Let, for example, $k = 13$. To compute x^{13}, we work as follows.

2.1 Define two classical registers. One will hold the base, and another will hold the running total. Initialize the base register to $x \pmod{n}$, and the running total to 1.
2.2 Find the binary representation of 13, which is 1101_2. We will have as many steps as there are numbers in the binary representation, in this case 4.
2.3 Examining the digits **right to left**, do the following:
2.3.1 Look at the digit.If the digit is 1, we multiply the running total by the base register and store the result in the running total. If the digit is 0, we do nothing.
2.3.2 If this is the last digit,terminate the algorithm.

2.3.3 Square the base register.

2.3.4 Repeat from 2.3.1 until there are no more digits left.

Below is pseudocode of the algorithm.

Algorithm 1 Modular Exponentiation

Input: Integers x, k, m

Output: $x^k \pmod{m}$

```
1: b ← x (mod m)
2: rt ← 1
3: while k > 0 do
4:     if k ≡ 1 (mod 2) then rt ← rt · b (mod m)
5:     end if
6:     k ← k/2 (integer division)
7:     if k = 0 then
8:         break;
9:     end if
10:    b ← (b · b) (mod m)
11: end while
12: return rt
```

This speeds up exponentiation modulo m from requiring $k - 1$ multiplications in the naive case, to $O(logk)$.

Now, this works fine for the 1-dimensional exponentiation case, but our accumulating step contains a product of multiple registers that may or may not contribute to the product. Suppose we have, e.g. $a_1 \ldots a_8$ in ascending order of value, and we want the product $a_1a_2a_3a_4a_5a_6a_7a_8$. The naive way of computing it would be to multiply a_1a_2, then multiply the product a_1a_2 with a_3 and so on. Regev proposes that it is much more efficient to do

$$((a_1a_8)(a_2a_7))((a_3a_6)(a_4a_5))$$

in a binary tree fashion, since you are always multiplying small numbers with small numbers.

Let us combine the two ideas into an algorithm that computes $\prod b_i^{e_i} \pmod N$. We will describe the way the algorithm would compute a product such as $a_1^{13}a_2^9a_3^3a_4^6 \pmod N$. The binary representations of the exponents are as follows $13 = 1101_2$ $9 = 1001_2$ $3 = 0011_2$ $6 = 0110_2$. This time,we will only use one register rt for the result, which we will initialize to 1.

Step 1. Examine the first bit. 13 and 9 have 1, so $rt \leftarrow a_1a_2 \cdot rt \pmod N$.
Step 2. Square the result,so $rt \leftarrow rt \cdot rt \pmod N$.
Current state of result: $rt = a_1^2a_2^2$.
Step 3. Examine the second bit. 13 and 6 have 1,so $rt \leftarrow a_1a_4 \cdot rt \pmod N$.
Current state of result:$rt = a_1^3a_2^2a_4$.
Step 4. Square the result.
Current state of result: $rt = a_1^6a_2^4a_4^2$.
Step 5. Examine the third bit. 3 and 6 have 1,so $rt \leftarrow a_3a_4 \cdot rt \pmod N$.
Current state of result:$rt = a_1^6a_2^4a_3a_4^3$.
Step 6. Square the result.
Current state of result: $rt = a_1^{12}a_2^8a_3^2a_4^6$.
Step 7. Examine the fourth bit. 13, 9 and 3 have 1,so $rt \leftarrow a_1a_2a_3 \cdot rt \pmod N$.
Current state of result: $rt = a_1^{13}a_2^9a_3^3a_4^6$.
Step 8. No more bits in the binary representation, so the algorithm terminates.

In our algorithm, we will want to produce every possible combination of exponents in superposition, which we will do by creating superpositions using controlled multiplications in each multi-exponentiation step where each base may or may not appear in the product - repeatedly applying

this procedure along with the squarings will give us all possible exponents in the end. We analyze the complexity of this procedure thoroughly in Chapter 14. For now, it is important to mention that we need to square $log_2 z$ times, where z is the largest exponent in the product. There is one multi-exponentiation step for each squaring step, and one more at the end, so $log_2 z + 1$ multi-exponentiations. From Regev's assumption, we get $log_2 z = O(d)$.

It should be noted here that most optimizations of Regev's algorithm that have come out to this day are concerned with optimizing this specific operation in terms of gates and/ or qubits. We describe the two most important ones in chapters 15 and 16.

We could optimize that by noticing that not every register needs to have size n. We can compute exactly how many qubits each of our product registers will need by simply summing the length of the respective multiplicands and allocate them accordingly. In Regev's algorithm, the bases we will use are the squares of the first d primes, and so, using the Prime Number Theorem, an upper bound for the total size of the first layer is

$$\#qubits < 2 \cdot 1.01624 \cdot p_d \cdot log_2 e$$

which can be rewritten as

$$\#qubits < 2.93 p_d \tag{7.1}$$

where p_d is the d-th prime number. Regev explicitly mentions this optimization in his paper, emphasizing the importance of multiplying small numbers with small numbers and how it affects space complexity due to the use of smaller registers, however the amount of register management required to perform multiplications of varied size in each step would make our code practically unreadable, hence we chose to leave this part for a followup paper. It does need to be said that this technique, according

to Regev in **Chapter 3 Equation 6** of [14], allows us to implement the multi-exponentiation circuit in $O(dlog^3 d)$.

Chapter 8

QFT and Gaussians

Earlier in the paper, we threatened the reader that we would describe how QFT in N dimensions would work in the context of Regev's algorithm. In this chapter, we will review this concept, along with how it is related to obtaining a basis for the dual lattice from cosets of the original lattice (which we can then use to obtain a basis for the original lattice itself).

The term $\mathbb{Z}_D^d$ will appear a lot in this chapter. This refers to the set of all d dimensional integer vectors $\mathbb{Z}^d$ whose entries are integers in the ring $\mathbb{Z}_D$ - the integers modulo D along with the addition operation. This means that vector addition is defined coordinate wise and modulo D.

Now, define the following gaussian function

$$\rho_s(\mathbf{e}) = exp(-\pi\|\mathbf{e}\|^2/s^2)$$

where $\mathbf{e}$ is the vector with entries e_i. The corresponding distribution has a standard deviation of s and a mean of $(max(e_i)+1)/2$. Fix some R to be our s.d., which we will tune later. It would be ideal if the mean in our gaussian was 0, for reasons that will become very clear later. To do that, let $D \in \{\, 2\sqrt{d}R, 4\sqrt{d}R \,\}$, and use $\mathbf{z} = \mathbf{e} - D/2$. This is also useful because quantum registers usually hold positive numbers, but our operations will require negative ones. Therefore, we will be mapping unsigned to signed integers. Then $\rho_R(\mathbf{z})$ is a gaussian with mean 0 and s.d. R. Taking D to be a power of 2, the vectors we are summing over have all integer entries and

thus their superposition forms a lattice.

For notation simplicity, we will write $K = \{ -D/2, \dots D/2 - 1 \}^d$. Rewriting $\rho_R(\mathbf{z})$ in terms of the shifted exponents, the following is a gaussian superposition over K.

$$|\psi\rangle = \sum_{\mathbf{z} \in K} \rho_R(\mathbf{z}) \, |\mathbf{z}\rangle \tag{8.1}$$

Regev himself showed a way to approximate such a superposition within $1/poly(d)$ in a previous paper [13]. Now, let's say we have a lattice $\Lambda \subset \mathbb{Z}_N^n$ and some coset of Λ $\Lambda_{\mathbf{t}} = \{ \Lambda + \mathbf{t} \}$ with representative $\mathbf{t}$, where $\mathbf{t} \in \mathbb{Z}^n$, but crucially $\mathbf{t} \notin \Lambda$. This represents Λ shifted by some arbitrary vector $\mathbf{t}$. Through some procedure which will be described in the actual algorithm chapter, suppose that this state is transformed to be proportional (equivalent up to normalization) to ψ_1, where

$$|\psi_1\rangle = \sum_{\mathbf{z} \in \Lambda_t \cap K} \rho_R(\mathbf{z}) \, |\mathbf{z}\rangle) \tag{8.2}$$

This represents a gaussian, shifted with respect to the representative t.

Now, suppose that we have the state ψ_2, where

$$|\psi_2\rangle = \sum_{\mathbf{z} \in \mathbb{Z}_D^d} \sum_{\mathbf{w} \in \Lambda_t \cap (\mathbf{z} + DZ^d)} \rho_R(\mathbf{w}) \, |\mathbf{z}\rangle \tag{8.3}$$

In essence, this describes a state where the gaussian wraps around modulo D, instead of containing shifted exponents. Considering our quantum registers can only hold strictly non-negative values, ψ_2 is essentially the state the actual system will be in.

Both ψ_1 and ψ_2 have 0 amplitude for non-Λ_t vectors, and then assign amplitudes to vectors with coordinates in their respective sets. For the state ψ_1, the amplitudes are dependent on vector length (which is generally what we desire), while for ψ_2 they are dependent on how large the coordinates are, up to a limit of D, where they wrap back around. This shows us that

those states are very similar, up to how many vectors where the individual coordinates aren't particularly close to D, but the overall length is large, are contained in each coset. Evidently, this scales with the number of dimensions - the more entries in the vector, the more coordinate partitions exist that sum up to large lengths. Regev proves with Claim A.5 that

$$\| |\psi_1\rangle - |\psi_2\rangle \| \leq 2^{1-d}$$

This tells us that for large values of d, these states are basically the same in terms of probability amplitudes, so applying transformations on either of them has the same effect.

Lastly, define the QFT over $\mathbb{Z}_D^d$, applied on some quantum state $|x\rangle$, as follows:

$$|\mathbf{x}\rangle \mapsto \frac{1}{\sqrt{D^d}} \sum_{\mathbf{y} \in \mathbb{Z}_D^d} e^{2\pi i \langle \mathbf{x}, \mathbf{y} \rangle / D} |\mathbf{y}\rangle \tag{8.4}$$

Let us now finally apply the operator from (8.4) to (8.2). This results in the following state $|\psi'\rangle$, where

$$|\psi'\rangle = \frac{1}{\sqrt{D^d}} \sum_{\mathbf{y} \in \mathbb{Z}_D^d} \sum_{\mathbf{z} \in \Lambda_t \cap K} \rho_R(\mathbf{z}) e^{2\pi i \langle \mathbf{z}, \mathbf{y} \rangle / D} |\mathbf{y}\rangle$$

Instead of writing the previous equation like that, note that since $\mathbf{z} \in \Lambda_t$, every $\mathbf{z}$ can be written as $\mathbf{z}_\Lambda + \mathbf{t}$ for some vector $\mathbf{z}_\Lambda$ in Λ. Now, by linearity of the inner product, we can write

$$\langle \mathbf{z}, \mathbf{y} \rangle = \langle \mathbf{z}_\Lambda, \mathbf{y} \rangle + \langle \mathbf{t}, \mathbf{y} \rangle$$

Here, we remind the reader that the probability of measuring a specific vector is equal to the magnitude of the square of the amplitude of said vector in the state, so $|A^2(\mathbf{y})|$ for some vector $\mathbf{y}$. Combining all of these, the amplitude $A(\mathbf{y})$ for any $\mathbf{y}$ in the state $|\psi'\rangle$ is

$$A(\mathbf{y}) = \frac{e^{\frac{2\pi i \langle \mathbf{t}, \mathbf{y} \rangle}{D}}}{\sqrt{D^d}} \sum_{\mathbf{z}_\Lambda \in \Lambda \cap K - \mathbf{t}} \rho_R(\mathbf{z}_\Lambda + \mathbf{t}) e^{2\pi i \langle \mathbf{z}_\Lambda, \mathbf{y} \rangle / D} \tag{8.5}$$

Suppose some vector $\mathbf{w} = \mathbf{y}/D$. By the definition of the inner product, $\langle \mathbf{z}_\Lambda, \mathbf{y} \rangle / D$ is an integer if and only if $\langle \mathbf{z}_\Lambda, \mathbf{w} \rangle$ is also an integer. To show that vectors satisfying this condition will also by definition be measured the most often, we can observe that the amplitude is maximal when $exp(2\pi i \langle \mathbf{z}_\Lambda, \mathbf{y} \rangle / D)$ is 1 across all inputs, since the gaussian is independent of $\mathbf{y}$. This is equivalent to the sum only containing vectors with amplitude 1. It is easy to see that this only happens when $\langle \mathbf{z}_\Lambda, \mathbf{y} \rangle / D$ is an integer. Furthermore, $\langle \mathbf{z}_\Lambda, \mathbf{w} \rangle \in \mathbb{Z}$ is exactly the definition of a dual lattice vector.

Now, regarding (8.5), note that the constant global phase outside the sum does not affect the measurement probability because it represents a vector of length 1 with only its phase being dependent on $\mathbf{y}$ ($|e^{\frac{2\pi i \langle \mathbf{t}, \mathbf{y} \rangle}{D}}| = 1$), and the denominator $\sqrt{D^d}$ is the same for all $\mathbf{y}$. It turns out that if $\|\mathbf{t}\| << R$, then the mean of the gaussian isn't affected much and we have

$$\rho_R(\mathbf{z}_\Lambda + \mathbf{t}) \approx \rho_R(\mathbf{z}_\Lambda)$$

This is precisely why we centered our gaussian at 0: now the smallest vectors will have the biggest weights.

Combining the two facts above, we get that $|A^2(\mathbf{y})|$ is maximal for short vectors of the dual lattice. In fact, the smaller R (the standard deviation) is, the shorter our vectors will end up being. It can't be too small though, else we might outweigh the exponential factor and get non-dual lattice vectors, purely because they are short. We will therefore be tuning R - Regev [14] claims that a value $R > \sqrt{2d}$ guarantees with almost certainty that we get dual lattice vectors.

Note that what we just described is a simplified explanation of the effect of the QFT on ψ_1. There was no mention of the general probability distribution of the outcome $A(y)$; rather, we just explained where the most likely amplitudes lie. However, the reader should not be concerned with the fact that the other $\mathbf{y}$ have nonzero amplitudes. In actuality, Regev [14] in

Appendix A shows that the probability distribution after applying QFT on the similar state ψ_2 is overwhelmingly concentrated on grid points closest to the dual lattice vectors. To prove that rigorously, he uses the Poisson summation formula and arrives at the result that the difference between the two states post QFT is on the order of $O(2^{-d})$. We chose to omit the formal proof in favor of the more intuitive explanation presented above.

Lastly, below we present pseudocode for the QFT over Z_D^d. We use the Hadamard gate H, which should be well known to the reader, and gates SWAP and CPR that perform the following state transitions:

SWAP gate:

$$|a\rangle\,|b\rangle \rightarrow^{SWAP} |b\rangle\,|a\rangle$$

CPR (Controlled Phase Rotation) gate on two qubits:

$$|a_k\rangle\,|b_j\rangle \rightarrow^{CPR} exp(2\pi i\frac{a_k b_j 2^{k+j}}{D})\,|a_k\rangle\,|b_j\rangle$$

Algorithm 5.3: Quantum Fourier Transform over Z_D^d

Input:Quantum registers consisting of $|e_1, e_2, \ldots, e_d\rangle$ in a superposition where the e_i are L qubit numbers, D.
Output: The result of the QFT in the same registers.
Let $e_{i,j}$ being the i_{th} qubit of the j_{th} register, from MSB to LSB.
for j from 1 to d **do**:
    for i from 1 to L **do**:
        $|e_{i,j}\rangle \rightarrow^{H} |e_{i,j}\rangle$
        for k from $i+1$ to L **do**
            $|e_{i,j}\rangle\,|e_{k,j}\rangle \rightarrow^{CPR(e_{k,j},e_{i,j})} (|e_{i,j}\rangle)\,|e_{k,j}\rangle$
        end for
    end for
    for i from 1 to $\lfloor L/2 \rfloor$ **do**
        $|e_{i,j}\rangle\,|e_{L-i+1,j}\rangle \rightarrow^{SWAP} |e_{i,j}\rangle\,|e_{L-i+1,j}\rangle$

13: **end for**

14: **end for**

Chapter 9

LLL

The final piece of mathematical theory we need to understand before we can analyze Regev's algorithm in a simple and coherent manner has to do with the Lenstra–Lenstra–Lovász algorithm, or LLL for short[1]. In reality, the original paper dealt with factoring polynomials with rational coefficients in polynomial time. However, the technique developed essentially converts polynomial factorization into a lattice problem. Before we explain the problem, it is best if we briefly go through the Gram-Schmidt orthogonalization process.

The classical Gram-Schmidt process is defined as an algorithm that performs the following operation:
Given a finite, linearly independent set of vectors

$$\mathbf{S} = \{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k\}$$

that span some k-dimensional subspace of $\mathbb{R}^n$ with $k \leq n$, Gram-Schmidt outputs a set

$$\mathbf{S}' = \{\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_k\}$$

such that all u_i are orthogonal to each other and $span(S) = span(S')$.
Now, let the Gram-Schmidt coefficients $\mu_{i,j}$ be equal to the following ratio:

$$\mu_{i,j} = \frac{\langle v_i, u_j \rangle}{\langle u_j, u_j \rangle}$$

[1]This is not a mistake,the name really refers to two people named Lenstra. Unfortunately, they are brothers.

Then the steps for Gram-Schmidt are as follows:
1.$\mathbf{u}_1 := \mathbf{v}_1$
2.For all i with $2 \leq i \leq k$:
3.Compute $\mu_{i,j}$ for j from 1 up to $i-1$
4.$\mathbf{u}_i := \mathbf{v}_i - \sum_{j=1}^{i-1} \mu_{i,j}\mathbf{u}_i$
5.End the loop $\mathbf{u_i}$
6.Normalize the $\mathbf{u_i}$ (compute the norm $\|\mathbf{u_i}\|$ and divide each coordinate of $\mathbf{u_i}$ by said norm.)

Now, we are ready to describe what LLL does. Firstly, we define an LLL-reduced basis as follows:

Definition 9.1 (LLL-reduced basis): Given a basis

$$\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_d\}$$

denote $\mathbf{B}' = \{\mathbf{b}'_1, \mathbf{b}'_2, \ldots, \mathbf{b}'_d\}$ as the basis that is produced by applying the Gram-Schmidt process on $\mathbf{B}$, and $\mu_{i,j}$ as the associated Gram-Schmidt coefficients. Then, the basis is LLL-reduced if there exists a parameter $\delta \in (1/4, 1]$ such that:

1.$|\mu_{i,j}| \leq 1/2$ for $1 \leq i < j \leq d$
2.$\delta\|\mathbf{b'}_{k-1}\|^2 \leq \|\mathbf{b}'_k\|^2 + \mu^2_{k,k-1}\|\mathbf{b}'_{k-1}\|^2$ for $2 \leq k \leq d$, the **Lovasz condition**.

In practice, bigger values of δ correspond to more orthogonal bases, but a longer algorithm. δ will therefore be a tunable parameter that will tell us how well the basis has been reduced. $\delta = 1$ corresponds to the shortest possible basis, but polynomial time complexity is guaranteed only when $\delta < 1$. We will be setting the parameter $\delta = 3/4$, as suggested by the authors of the LLL paper.

The LLL algorithm aims to compute an LLL-reduced basis that is more orthogonal than the input and nearly as short as possible with respect to

Euclidean length, while still spanning the same lattice.

Below, we describe pseudocode for LLL.

Algorithm 2 δ-LLL Algorithm

Input: Lattice basis $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_d\}$, parameter $\delta \in (1/4, 1)$

Output: δ-LLL reduced basis

1: Compute $\mathbf{B}' = \{\mathbf{b}'_1, \mathbf{b}'_2, \ldots, \mathbf{b}'_d\}$ and coefficients $\mu_{i,j} = \frac{\langle \mathbf{b}_i, \mathbf{b}'_j \rangle}{\langle \mathbf{b}'_j, \mathbf{b}'_j \rangle}$
2: $k \leftarrow 2$
3: **while** $k \leq d$ **do**
4: **for** $j = k - 1$ to 1 **do**
5: **if** $|\mu_{k,j}| > 1/2$ **then**
6: $m \leftarrow \text{round}(\mu_{k,j})^2$
7: $\mathbf{b}_k \leftarrow \mathbf{b}_k - m\,\mathbf{b}_j$
8: Repeat step 1 for the new values,updating $\mathbf{B}'$ and $\mu_{i,j}$
9: **end if**
10: **end for**
11: **if** $\|\mathbf{b}'_k + \mu_{k,k-1}\mathbf{b}'_{k-1}\|^2 \geq \delta \|\mathbf{b}'_{k-1}\|^2$ **then**
12: $k \leftarrow k + 1$
13: **else**
14: Swap $\mathbf{b}_k$ and $\mathbf{b}_{k-1}$
15: $k \leftarrow \max(k - 1, 2)$
16: Recompute $\mathbf{b}^*_i$ for $i = k, k + 1, \ldots, d$ and $\mu_{i,j}$ for $j < i$
17: **end if**
18: **end while**
19: **return** $\mathbf{B}_{LLL} = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_d\}$

Essentially, LLL does two things repeatedly:

1.Reduces the **size** of each basis vector by removing unnecessary components along earlier ones,until the factor μ is sufficiently small.

2. Reorders the vectors so that shorter vectors appear earlier.

We can think of step 7 as essentially subtracting all projections of the

k-th vector onto previous vectors, therefore attempting to minimize the component of $\mathbf{b}_k$ along the direction of a previous vector. If the factor of that projection is less than $1/2$, meaning the vector component of $\mathbf{b}_k$ along a given direction that already includes a vector in the basis is less than $1/2$, then this is considered sufficient.

Now,to understand why the actual algorithm works, let us consider a quantity which we will call the potential function $D(d)$. It is defined as

$$D(d) = \prod_{i=1}^{d} \|\mathbf{b}_i'\|^{2(d-i-1)}$$

$Vol(\Lambda)$ as the **volume** of the lattice Λ. The volume of a lattice refers to a quantity that measures the density of the lattice points in space, and is defined as

$$Vol(\Lambda) = \sqrt{det(B^T B)} \tag{9.1}$$

where B is the $n \times d$ whose columns are the $\mathbf{b}_i$. For full rank lattices where B is a $n \times n$ matrix, $Vol(\Lambda)$ is **equivalent** to the determinant of B $\mathbf{det}(\mathbf{B})$. We will not demonstrate the proof here,but an equivalent definition for $Vol(\Lambda)$ is the following:

$$Vol(\Lambda) = \prod_{i=1}^{d} \|\mathbf{b}_i'\| \tag{9.2}$$

It should be noted that an interpretation of the 2-D determinant is that it represents the area of the parallelogram formed by the basis vectors. In n dimensions,the parallelogram becomes a parallelepiped formed by the n basis vectors,and area corresponds to volume. In general, Vol can be thought of as a generalization of the determinant when it is possible that there are not enough vectors to span the whole space, so the volume of the parallelepiped formed by whatever vectors exist in our B.

Suppose we restrict the value computed in (9.2) to being the product of only j terms, with $j < d$. Then let $Vol_j(\Lambda)$ be the volume of the parallelepiped formed by the first j basis vectors.

This means that the i-th term in the $D(d)$ product essentially represents the product squared volumes of the parallelepipeds formed by the first i basis vectors, so $(Vol_i(\Lambda))^2$. The value for the exponent is simply a weight with respect to the order of the vectors: earlier vectors and their parallelepiped volumes affect the value more.

LLL's termination condition is when the vectors are in ascending order of size, and every 2 consecutive vectors satisfy the Lovasz condition. If the orthogonal component of a given vector is smaller than the one of the previous vector,the vectors are swapped, and we recompute their orthogonal components. We can prove that this action, together with the Lovasz condition, guarantee a decrease of the potential function after every swap. Eventually, however, after finitely many swaps, $D(d)$ will hit a local minimum, as there will be no further swaps to be made.This is when the algorithm terminates, producing a sufficiently small basis for Λ.

We will now describe a very important bound for the LLL basis that has been proven to hold.
Let $\mathbf{b}_1$ be the first (and therefore shortest) vector in the LLL basis. Also let $m(\Lambda)$ be the length of the smallest nonzero vector of Λ. In mathematical terms,

$$m(\Lambda) = min\{\, \|\mathbf{x}\|^2 \, s.t. \mathbf{x} \in \Lambda, \mathbf{x} \neq \mathbf{0} \,\}$$

Then, the following bound holds:

$$\|\mathbf{b}_1\| \leq 2^{(d-1)/2} \|m(\Lambda)\| \tag{9.3}$$

This essentially states that the first vector is not much larger than the shortest vector of the lattice. This is important because the reason we are using LLL in our larger scheme is not to get a LLL-reduced basis, but rather to solve SVP to some approximation factor. By taking every vector in the LLL-reduced basis that satisfies our bound, LLL inadvertedly provides a

polynomial-time solution to this problem. Note however that for $\delta < 1$, LLL isn't guaranteed to terminate in polynomial time. SVP is, after all, NP-Hard.

It should be mentioned that LLL's runtime scales inversely with the length of the largest input vector. For this reason, we prefer inputting shorter vectors into LLL. One got therefore get a sense of the steps we will be performing in Regev's algorithm by immediately thinking back to the gaussian in the previous chapter and how it increased the probability of shorter vectors.

Lastly, we need to explain why we are even trying to find a reduced basis in the first place. In Chapter 6, we talked about "making sure the vectors are as short as possible". Let us forget the modular world for a second - after all, LLL tries to minimize the size of the vectors, not the size of the product. Consider the equations $X \equiv 1 \pmod{N}$ This is equivalent to saying

$$X = mN + 1$$

for some $m \in \mathbb{Z}$. This equation is minimized at $|m| = 1$ ($m = 0$ would correspond to the zero vector, but LLL by definition only finds non-zero vectors). The smallest possible values for the trivial solutions therefore correspond to the values $|X| = N + 1$ and $|X| = N - 1$. On the contrary, the non-trivial solutions satisfy the equation

$$X^2 = mN + 1$$

The smallest possible values for the non-trivial solutions correspond to some value $X \approx \sqrt{N}$, which is much closer the size we expect for our p and q. Since the bases stay constant, the value of X depends on how short the exponent vector is. This means that shorter exponent vectors are more likely to correspond to non-trivial solutions. In fact, the bound Regev talks about, e.e. $\|\mathbf{z}\| \leq 2^{O(d)}$ is directly derived from the bound in (9.3).

Evidently, assuming we construct the right lattice to account for noise, we can run LLL on it to get our desired result.

Chapter 10

More lattices

After reading the LLL chapter, the reader might have possibly come to the conclusion that Regev collects a single sample - the first vector from LLL - from every run of the algorithm, and hopes that this vector happens to correspond to a non-trivial root. This is, however, not the case. Certain procedures we've described up to now, like the gaussian superposition and the chance of hitting a trivial root rely on probabilistic outcomes, and we also need to account for quantum noise. Regev therefore attempts to generate as many vectors as possible from a single run, to increase his probability of finding non-trivial roots.

Now, suppose we have our $d + k$ normalized d-dimensional samples of Λ^*. By normalized, we mean that we have divided the original samples by D and now every entry is in $[0, 1)$, so we've mapped every sample to a unique point on the unit torus. Suppose now that each sample corresponds to a real Λ^* vector $\mathbf{v_1}, \mathbf{v_2}, \ldots, \mathbf{v_{d+k}}$ multiplied by some additive noise δ_i for each $\mathbf{v_i}$. Let δ be some upper bound on the magnitude of the δ_i, and define the $2d + k$ dimensional lattice Λ' spanned by the columns of M, where M is the following matrix (we need $k \geq 4$ from Pomerance)

$$M = \left[\begin{array}{c|c} I_{d\times d} & 0 \\ \hline \delta^{-1}\mathbf{v_1^T} & \\ \vdots & \delta^{-1} \cdot I_{k\times k} \\ \delta^{-1}\mathbf{v_{d+k}^T} & \end{array}\right] \tag{10.1}$$

Regev proves using the Cauchy-Schwarz inequality the following lemma (combination of Lemma 4.4 and Corollary 4.5 in [14]).

Lemma 10.1: For any $\mathbf{u} \in \Lambda$, there exists a vector $\mathbf{u}' \in \Lambda'$ whose first d coordinates are equal to $\mathbf{u}$ with norm $\|\mathbf{u}'\| \leq \|\mathbf{u}\| \cdot \sqrt{1+d+k}$. Moreover, for $k = 4$ it holds that the first d coordinates of every nonzero column vector $\mathbf{u}' \in \Lambda'$ with norm

$$\|\mathbf{u}'\| < \frac{\delta^{-1}}{6 \cdot (4det(\Lambda))^{1/(d+4)}}$$

form a vector in Λ with probability at least $1/4$.

To understand why we are doing this, suppose we multiply some arbitrary $\mathbf{u} \in \mathbb{Z}^d$ with the first d columns of M. This results in a $2d+k$ dimensional vector $\mathbf{u}'$ whose first d coordinates are the first d coordinates of $\mathbf{u}$ and the next $d+k$ coordinates are inner products of some Λ^* vector scaled by δ^{-1} with some $\mathbb{Z}^d$ vector, so they're really close to some integer multiplied by δ^{-1}. Now, suppose we define a vector $\hat{\mathbf{u}}$ that is equal to subtracting the closest integer to $\langle \mathbf{u}, \mathbf{v_m} \rangle$ from each of the last m coordinates of u' (the ones that don't contain $\mathbf{u}$. Then the last m coordinates of $\hat{\mathbf{u}}$ are either 0 if $\langle \mathbf{u}, \mathbf{v_m} \rangle = 0$, or δ^{-1} if not, so 0 if u is in the dual lattice Λ^* (e.e Λ) and δ^{-1} otherwise.

Since $B\hat{\mathbf{u}}$ is part of the span of the vectors formed by the columns of B, finding the shortest vector in that span corresponds to finding the $\hat{\mathbf{u}}$ with the smallest norm. Moreover, since the last m coordinates of $\hat{\mathbf{u}}$ will be nearly zero, we are just looking for the shortest $\mathbf{u}$, and the process of finding short vectors will automatically filter out vectors not in Λ by virtue of them having a squared sum of coordinates $O(k\delta^{-2})$ larger than vectors in Λ (essentially, we are looking for the shortest vectors in Λ). This means that running LLL on M will provide us with at least one vector whose first d coordinates correspond to a short vector in Λ.

For now, from this correspondence, we can see how from our noisy samples of Λ^* we will produce a clean, short base for Λ sufficient to give us a non-trivial factor. Suppose we run LLL on the basis formed by M, returning some vectors $\mathbf{b_1}, \mathbf{b_2}, \ldots, \mathbf{b_{d+k}}$ and then Gram-Schmidt orthogonalization on the result, giving us some orthogonal vectors $\mathbf{b_1^*}, \mathbf{b_2^*}, \ldots, \mathbf{b_{d+k}^*}$. Let T be some norm bound, and let $h \leq k$ be the largest number such that

$$\|\mathbf{b_h^*}\| < 2^{k/2}T \tag{10.2}$$

A corollary of the Lovasz condition can help tighten this bound to[1].

$$\|\mathbf{b_h^*}\| < \sqrt{k}2^{k/2}T \tag{10.3}$$

Then, according to Claim 5.1 in Regev's paper, any vector in the sublattice $\Lambda'_T \subseteq \Lambda'$ formed by vectors of norm $\leq T$ can be generated by $\mathbf{b_1}, \mathbf{b_2}, \ldots, \mathbf{b_h}$. Due to **Lemma 10.1**, the first d coordinates of the $\mathbf{b_i}$ correspond to vectors in Λ. This gives us not just one, but multiple short vectors in Λ that could also be in Λ/Λ_0. In fact, at least one of those vectors is guaranteed to not be in Λ_0, otherwise Λ_T would be equal to Λ_0 itself (since Λ_0 is a group with closure). Therefore, taking $k \geq 4$ (to overshoot the >1/4 probability that each one of them is in Λ_0), we can reliably end up with a vector corresponding to a non-trivial factor.

Lastly, note that the reason we normalized our samples is that it really doesn't matter how large the original sample is. It is very easy to prove that the inner product of any arbitrary vector with a normalized vector of the dual is still an integer, and thus our previous logic still holds.

[1] It should be obvious that all b_i^* with $i \leq h$ also obey this bound.

Part III

Regev's Algorithm

Chapter 11

Presenting the algorithm

Regev's algorithm can therefore be summarized in the following 12 steps. Let some $d \in \mathbb{Z}$ with $d \approx \sqrt{n}$.

1. Create a superposition of all d-dimensional vectors $e_1 \dots e_d$, and store them into d quantum registers, weighted by the gaussian function described in Chapter 8.
2. Create a quantum register which will contain the superposition $\prod_{i=1}^{d} a_i^{e_i} \pmod N$, where a_i are the squares of the first d primes, using the procedure described in Chapter 7.
3. Measure the product register to obtain some value u. This forces the d exponent registers to collapse into a superposition of vectors that satisfy $\prod_{i=1}^{d} a_i^{e_i} = u$. This represents some coset Λ_t of the lattice Λ formed by the vectors that satisfy the above relation for $u = 1$ (Chapter 5).
4. Apply QFT to the vector registers. This turns the superposition of the d registers into a superposition of dual lattice vectors, and the representative t into a phase applied to the dual lattice vectors.
5. Measure the vector registers, obtaining a d-dimensional vector, and divide the result by D to get a sample of Λ^*.
6. Repeat steps 1 through 5 $d + k$ times to get $d + k$ samples and use them to create a basis of Λ', akin to the procedure described in Chapter 10.
7. Solve SVP on the basis of Λ' using LLL, acquiring vectors $\mathbf{e}', \mathbf{e}'', \mathbf{e}''', \dots$

etc. with coordinates $e'_1 \dots e'_d, e''_1 \dots e''_d$ etc. that satisfy the bound from Chapter 10.

8.For each candidate $\mathbf{e} \in \{ \mathbf{e}', \mathbf{e}'', \mathbf{e}''', \dots \}$ in order, do the following

9.Compute $X = \prod_{i=1}^{d} b_i^{e_i} \pmod{N}$

10. Compute $\gcd(X - 1, N)$ to get a factor of N.

11. If the factor is trivial, try the next candidate.

12. If all factors are trivial, run the algorithm again.

Chapter 12

Pseudocode

Algorithm 5.2: Regev's Algorithm

Input: A composite number N, where $N = p \cdot q$, R, T, D
Output: primes p and q
$n \leftarrow \lceil \log_2 N \rceil$ ▷ Classical preprocessing
$d \leftarrow \lceil \sqrt{n} \rceil$
$B[] \leftarrow$ the first d primes, in order
$i \leftarrow 0$ ▷ Quantum
while $i < d$ **do**
    Prepare the gaussian. Suppose *amps* represents the amplitudes
    Define two quantum registers, $e[size]$ and *prod*
    $e \leftarrow amps[e_i]$
    $prod \leftarrow \prod_{i=1}^{d} B[i]^{2e_i}$ ▷ Modular Exponentiation
    $u = measure(prod)$ ▷ This changes the amplitudes in e.
    $e \leftarrow QFT(e)$
    $L_i^* = measure(e)/D$
    $i \leftarrow i + 1$
end while ▷ The rest of the algorithm is purely classical
Form the matrix M from (10.1)
$M' = LLL(M)$
$M'' = GramSchmidt(M')$
Iterate through M'' and find the first index h where $\|\mathbf{v_{h+1}}\| >= \sqrt{k}2^{k/2}T$

21: SV = The list of all vectors $\mathbf{u_1}, \mathbf{u_2}, \ldots, \mathbf{u_h}$ in M'
22: **for** $SV(j)$ in SV **do**
23: $X \leftarrow \prod_{i=1}^{d} B[i]^{SV_j(e_i)}$
24: $l \leftarrow \gcd(X-1, N)$
25: **if** $l \neq 1$ and $l \neq N$ **then**:
26: **return** l
27: **end if**
28: **end for**
29: **return** 0

Our algorithm implementation will also be based on the following circuits, which we will treat as black boxes:

Modular Adder (Out-of-Place):

$$|a\rangle\,|b\rangle\,|0^n\rangle \rightarrow |a\rangle\,|b\rangle\,|a+b \pmod N\rangle$$

Modular Multipier (Out-of-Place):

$$|a\rangle\,|b\rangle\,|0^n\rangle \rightarrow |a\rangle\,|b\rangle\,|a\cdot b \pmod N\rangle$$

Modular Squarer (Out-Of-Place) [1]

$$|a\rangle\,|0^n\rangle \rightarrow |a\,|a^2 \pmod N\rangle\rangle$$

[1] Could be implemented using the modular multiplier.

Chapter 13

Qiskit and complexity

The program code is available on github.

Notice that the D/2 shift doesn't appear anywhere in our code, besides determining the amplitudes for the gaussian. That is because in practice, we don't need to implement an actual shift: the product is computed using the shifted indices, the QFT ignores the shift, and the matrix M is simply normalized to [0,1).

Most of the gate implementations were borrowed from [11], as it turned out to be the only (to our knowledge) reliable source outlining the way such a circuit could be implemented. The authors attempt to avoid in-place quantum-quantum multiplication in the multi-exponentiation step, due to the heavy gate count. Following their example, we also refrain from using it. We retain the following notation from this paper:

$|a\rangle$ is some arbitrary state.
$|0^{cn}\rangle$ refers to $c \cdot n$ qubits initialized to the 0 state.
$|g^{n}\rangle$ refers to n dirty ancilla qubits initialized to some arbitrary state.

Let us describe the modular exponentiation procedure. The main ingredient we will use to perform modular multi-exponentiation is the function MOD-PROD.

MOD-PROD executes the following state transition:

$$|a\rangle |a^{-1}\rangle |b\rangle |b^{-1}\rangle |g^n\rangle |0^{5n}\rangle \rightarrow |a\rangle |a^{-1}\rangle |ab\rangle |(ab)^{-1}\rangle |g^n\rangle |0^{5n}\rangle \tag{13.1}$$

Notice also what happens when we apply MODPROD to the new state with the first two input registers flipped:

$$|a^{-1}\rangle |a\rangle |ab\rangle |(ab)^{-1}\rangle |g^n\rangle |0^{5n}\rangle \rightarrow |a^{-1}\rangle |a\rangle |b\rangle |b^{-1}\rangle |g^n\rangle |0^{5n}\rangle$$

This shows us that we can recursively uncompute our exponentiation tree by repeatedly applying this operation.

MOD-PROD makes 6 calls to the MUL-ADD-MOD circuit and two swaps. MUL-ADD-MOD performs the following state transition:

$$|a\rangle |b\rangle |t\rangle |0^{2n+A}\rangle \rightarrow |a\rangle |b\rangle |(t + ab) \mod N\rangle |0^{2n+A}\rangle \tag{13.2}$$

We will use this circuit as a black box, as Ragavan and Viswantanathan do. A potential implementation makes two calls to an out-of-place modular multiplication circuit and one to an in-place modular adder, so multiply -> add -> invert multiplication. The out-of-place modular multiplication circuit performs

$$|a\rangle |b\rangle |0^{n+A}\rangle \rightarrow |a\rangle |b\rangle |(ab) \mod N\rangle |0^{A}\rangle \tag{13.3}$$

(more words on such a circuit below), while the in-place modular adder performs

$$|a\rangle |b\rangle |0^{n}\rangle \rightarrow |a\rangle |(a + b) \mod N\rangle |0^{n}\rangle \tag{13.4}$$

The circuit from (13.4) could be implemented in the fashion outlined in [16] using addition and reduction, or using a QFT-based adder, while the one in (13.3) can be customized to utilize any multiplication algorithm to adjust the qubit/gate tradeoff.

Our binary tree in at first initialized using d control qubits in superposition to either $|1\rangle$ or $|b_i\rangle$. This means that b_i either participates in the product

or doesn't. Those qubits are essentially the $i-th$ bit from each of our exponent registers, so for each multiplication step we take the next qubit, which at the end gives us a superposition of all numbers up to 2^d in every register (since we don't need to go up to 2^n to generate sufficient exponents due to Regev's assumption).

We also keep a list of registers containing the multiplicative inverses of each b_i mod N initialized with the same control qubit as the original b_i
To do this, we use the CMMC gate, which works as follows:
Suppose we have a control qubit $|ctrl\rangle$ in a Hadamard superposition, a n-bit register $|a\rangle$ initialized to the 1 state and a classical constant c. We calculate the bitmask $bm = 1 \oplus c$, and then, for every bit bm_j in bm, if $bm_j == 1$, we apply a cx gate with $ctrl$ as control and bit a_j of $|a\rangle$ as target. This is effectively a controlled multiplication at the end of which $|a\rangle$ will be in a equal superposition of 1 and c. Accordingly, at the end of the circuit, we use the same gate to uncompute the initial registers, using our precomputed list of inverses.

Let us outline below the state transitions that will happen during a factor multiplication step. Suppose $d = 4$, and let a_{ij} denote the j-th bit of the i-th base register. Assume again that all multiplications are mod N. Assume also that we have the accumulator register $|acc\rangle$ along with its inverse $|acc^{-1}\rangle$. Assume also that we have precomputed the multiplicative inverses of each a_i mod N, denoted a_i^{-1}.

1.Apply Hadamard gates to d qubits initialized in the 0 state to put them in superposition.

2.For each d_i of the d Hadamard qubits, apply CMMC with d_i as control and a_{ij} as target constant and again apply CMMC with d_i as control and a_{ij}^{-1} as target constant for every j with $0 \leq j < n$. Now we have $2d = 8$

quantum registers each pair of which may or may not contain a base and its inverse.

4.$|a_1\rangle\,|a_1^{-1}\rangle\,|a_4\rangle\,|a_4^{-1}\rangle\,|a_2\rangle\,|0^{5n}\rangle \rightarrow |a_1\rangle\,|a_1^{-1}\rangle\,|a_1a_4\rangle\,|(a_1a_4)^{-1}\rangle\,|a_2\rangle\,|0^{5n}\rangle$ - a_2 plays the role of the dirty ancilla g here.

5. $|a_2\rangle\,|a_2^{-1}\rangle\,|a_3\rangle\,|a_3^{-1}\rangle\,|a_1\rangle\,|0^{5n}\rangle \rightarrow |a_2\rangle\,|a_2^{-1}\rangle\,|a_2a_3\rangle\,|(a_2a_3)^{-1}\rangle\,|a_1\rangle\,|0^{5n}\rangle$ - a_1 plays the role of the dirty ancilla g here.

6.$|a_1a_4\rangle\,|(a_1a_4)^{-1}\rangle\,|a_2a_3\rangle\,|(a_2a_3)^{-1}\rangle\,|a_1\rangle\,|0^{5n}\rangle \rightarrow$
$|a_1a_4\rangle\,|(a_1a_4)^{-1}\rangle\,|a_1a_2a_3a_4\rangle\,|(a_1a_2a_3a_4)^{-1}\rangle\,|a_1\rangle\,|0^{5n}\rangle$. Keep in mind that $|a_1a_2a_3a_4\rangle$ is essentially a superposition of all possible products between those 4 numbers - a_i simply represents what is contained in the register, which could be 1 or a_i.

7. $|a_1a_2a_3a_4\rangle\,|(a_1a_2a_3a_4)^{-1}\rangle\,|acc\rangle\,|acc^{-1}\rangle\,|a\rangle\,|0^{5n}\rangle \rightarrow$
$|a_1a_2a_3a_4\rangle\,|(a_1a_2a_3a_4)^{-1}\rangle\,|acc\cdot a_1a_2a_3a_4\rangle\,|(acc\cdot a_1a_2a_3a_4)^{-1}\rangle\,|a\rangle\,|0^{5n}\rangle$. Now, we uncompute:

8.$|(a_1a_4)^{-1}\rangle\,|a_1a_4\rangle\,|a_1a_2a_3a_4\rangle\,|(a_1a_2a_3a_4)^{-1}\rangle\,|a_1\rangle\,|0^{5n}\rangle \rightarrow$
$|(a_1a_4)^{-1}\rangle\,|a_1a_4\rangle\,|a_2a_3\rangle\,|(a_2a_3)^{-1}\rangle\,|a_1\rangle\,|0^{5n}\rangle$

9. $|a_1^{-1}\rangle\,|a_1\rangle\,|a_1a_4\rangle\,|(a_1a_4)^{-1}\rangle\,|a_2\rangle\,|0^{5n}\rangle \rightarrow |a_1^{-1}\rangle\,|a_1\rangle\,|a_4\rangle\,|a_4^{-1}\rangle\,|a_2\rangle\,|0^{5n}\rangle$

10.$|a_2^{-1}\rangle\,|a_2\rangle\,|a_2a_3\rangle\,|(a_2a_3)^{-1}\rangle\,|a_1\rangle\,|0^{5n}\rangle \rightarrow |a_2^{-1}\rangle\,|a_2\rangle\,|a_3\rangle\,|a_3^{-1}\rangle\,|a_1\rangle\,|0^{5n}\rangle$

Now the registers are reset and ready to reuse for the next iteration, and we have modified the accumulator with our result.

Evidently, since we are uncomputing everything at each iteration, we only need to use the d n-bit original base registers along with their inverses to compute the product. We also need d exponent controls for each of our d multiplications, for a total of $d^3 = n^{3/2}$ qubits. Our implementation of MOD-PROD will also require at least $3n + S$ clean ancilla qubits, where S is a parameter that will depend on our multiplication circuit and n dirty ones (we can ignore the latter cost by using other registers from our circuit).

Therefore, our multi-exponentiation circuit requires space (in the worst case where d is a power of 2) $2\sqrt{n} \cdot n + \sqrt{n} + 2n + A = 2n^{3/2} + 4n + \sqrt{n} + S = O(n^{3/2})$ qubits, where all qubits that don't concern the bases or the exponents will be uncomputed and reused in every multiplication step. Note that implementing the suggestion at the end of Chapter 7 would reduce the dominant term by allowing us to perform multi-exponentiation in $O(d \cdot log^3 d)$ according to Regev - over d times that amounts to $O(n \cdot log^3 n)$ which is an asymptotic improvement. We will see that it this won't matter in the end, since modular squaring raises our complexity to $O(n^{3/2})$ nonetheless, but it is important to note that this is a complexity analysis of the specific designs presented, and not of Regev's algorithm as a whole, albeit our implementation is designed to match asymptotically.

For the modular squaring part, [11] uses the MOD-PROD gate yet again. However, we want to maintain our binary representation for clarity [1], and because Ragavan's implementation isn't the primary subject for this paper. MOD-PROD still gave us good results for multi-exponentiation, but its main design was to implement reversible squaring according to Ragavan's method, which we can't do here. The goal is, as we referenced in the previous chapter, to implement a circuit performing

$$|a\rangle |0^n\rangle \rightarrow |a\, |a^2 \pmod N\rangle\rangle \tag{13.5}$$

Rines and Chuang [15] show us an alternate way we can do this using $2n + O(logn) = O(n)$ qubits and $O(n^2)$ gates (estimate from page 17 of [11]) combining QFT multiplication along with Montgomery reduction. This is essentially describing a specific implementation of the circuit in **Eq 13.3**. To remain consistent, due to the fact that the aforementioned paper suggests multiple ways of implementing such a circuit, we omit the implementation of this circuit in our code, instead treating modular

[1] Ragavan and Vaikuntanathan use a Fibonacci base, more on that in Chapter 13

squaring as a black box. We will, however, use those specific requirements to calculate the potentialcomplexity of our implementation.

As we established at the end of Chapter 7, such a circuit will be performed $O(d) = O(n^{1/2})$ times, leaving behind an n-bit register each time, for a total of $O(n^{3/2})$ qubits in circuit depth and space. In total, together with the qubit requirements from the multi-exponentiation, and the n-bit accumulator, it turns out that our qubit requirement for Regev's algorithm is in the order of $O(n^{3/2})$. In terms of gate count, we saw that our bottleneck is squaring, since it forces us to use a specific multiplication algorithm, while for general multiplication we can pick whichever algorithm we want for the 13.3 circuit. Our total gate count counts ends up being dominated by the factor of $O(n^2)$ squarings that are used $O(d) = O(n^{1/2})$ times, for a total of $O(n^{5/2})$ gates, better than Shor's $O(n^3)$. Lastly, by our results in Chapters 6 and 9, setting $k = 4$ gives us a probability that is good enough for all practical purposes. In general, we need $O(d)+$ (some constant number of runs) to have reliable results, so $O(\sqrt{n})$ runs. Summarizing:

Space: $O(n^{3/2})$
Gates: $O(n^{5/2})$
Independent runs: $O(\sqrt{n})$ with high probability

which matches Regev's complexity claims from [14]

Chapter 14

Further optimizations

The implementation we presented in Chapters 10 and 11 concerns mostly the algorithm as it was originally published, along with some necessary post-original paper notes regarding the number theoretical assumption. However, this thesis would be incomplete without including the new ideas that have been presented after the original paper regarding how the algorithm could be optimized even further in terms of qubits, noise, and gate count.

Chapter 15

Regev optimizations: Fibonacci exponentiation

The first optimization we are going to talk about has to do with transforming the base in which we represent the numbers in a base that is perhaps more suited when dealing with exponents. To do this, we will entice the amateur mathematics enthusiast by bringing the Fibonacci numbers and Fibonacci sequence into the picture. To explore this concept, first we will need to talk about Zeckendorf's theorem. The statement is the following:

Lemma 15.1 (Zeckendorf) :
Every positive integer k can be uniquely represented as a sum of non-consecutive Fibonacci numbers.In other word, for every $k \in \mathbb{Z}$:

$$k = \sum_j c_j Fi,\ c_j \in \{\, 0, 1 \,\} \tag{15.1}$$

No two consecutive c_i can be 1, because consecutive Fibonacci numbers can be replaced with the next Fibonacci number according to the recursive formula $F_n = F_{n-1} + F_n - 2$. This means that representing exponents as bits using a Fibonacci base is perfectly valid. We will now describe a way of computing a^k using a Fibonacci base. First, we need to compute the Fibonacci representation of k. To do that in a quantum circuit, we use the following greedy algorithm.

Step 1. Compute all Fibonacci numbers up to k using the recursive formula and store it in an array F, starting from $F[0] = 1, F[1] = 2$.
Step 2. Initialize an array F_b of 0s with size $m - 1$, where F_m is the largest Fibonacci number smaller than k
Step 3. Let $j \leftarrow m - 1$.
Step 4. $F[j] \leftarrow 1$
Step 5. $k \leftarrow k - F_m$
Step 6. Run binary search on F to compute the largest Fibonacci number smaller than k
Step 7. $F_m \leftarrow$ the value computed in Step 6
Step 8. $j \leftarrow$ the index of that value in F
Step 9. Repeat from Step 2 until $k = 0$.

F contains the binary representation of k. Now, computing a^k is very easy, because we can use the same logic as modular exponentiation.

Assume the greedy algorithm in the previous step leaves us with a Fibonacci representation k_ϕ consisting of all the c_j, and m Fibonacci numbers computed, starting from $F[0] = F_2 = 1$. Let us present the algorithm for Fibonacci exponentiation, as described by Kaliski [2]

Algorithm 3 Fibonacci exponentiation

Input: a, k, N, the sequence $c_1, c_2 \ldots c_m$

Output: $a^k \pmod N$

1: $(c, d, e) \leftarrow (1, a, 1)$
2: **for** $j = 1$ to m **do**
3: $\quad (c, d) \leftarrow (d, cd \pmod N)$
4: $\quad$ **if** $c_j == 1$ **then**
5: $\quad\quad e \leftarrow de \pmod N$
6: $\quad$ **end if**
7: **end for**

Consider the original Regev product $\prod_{i=1}^{d} b_i^{e_i}$ (the algorithm is also compatible with the $D/2$ shift). Suppose we have a Fibonacci representation of e_i

$$e_i = \prod_{j=1}^{m} c_{i,j} F_j \tag{15.2}$$

where $c_{i,j}$ denotes each of the earlier c_j but now for every base. Then we can plug (15.2) in the place of each exponent and write the resulting expression - which is **equivalent** to the Regev product, as follows:

$$\prod_{j=1}^{m} (\prod_{i=1}^{d} b_i^{c_{i,j}})^{F_j}$$

Denote the inner product as C_j. This leaves us with

$$\prod_{j=1}^{m} C_j^{F_j} \tag{15.3}$$

What we need to do here essentially is compute this over all possible C_j in superposition, raise C_j to the power of the next Fibonacci number, and do this over and over until $F_j > 2^d$. To compute the C_j, we can basically do exactly what we did in Chapter 13 for modular multi-exponentiation - precompute the inverses, build the tree, multiply into the accumulator, uncompute the tree.

Where this logic shines however, is with regards to squaring. In our implementation, we had to use a different algorithm for squaring. However, we don't actually need to square when representing numbers in the Fibonacci base. Consider what happens if we sequentially applied our MOD-PROD gate to some arbitrary $|a\rangle\,|a\rangle$:

1.$|a\rangle\,|a^{-1}\rangle\,|a\rangle\,|a^{-1}\rangle \to |a\rangle\,|a^{-1}\rangle\,|a^2\rangle\,|(a^2)^{-1}\rangle$
2.$|a\rangle\,|a^{-1}\rangle\,|a^2\rangle\,|(a^2)^{-1}\rangle \to |a^3\rangle\,|(a^3)^{-1}\rangle\,|a^2\rangle\,|(a^2)^{-1}\rangle$
3.$|a^3\rangle\,|(a^3)^{-1}\rangle\,|a^2\rangle\,|(a^2)^{-1}\rangle \to |a^3\rangle\,|(a^3)^{-1}\rangle\,|a^5\rangle\,|(a^5)^{-1}\rangle$

Notice the Fibonacci sequence appearing in the exponents! Now we have a reversible base accessing operation that doesn't leave us with dirty qubits in every iteration. More importantly, it seems that this can actually decrease the qubit count using the optimized multi-exponentiation idea from Chapter 7 with the reduced register sizes, because we won't need the extra $O(n^{3/2})$ qubits to perform squaring anymore.

Let us, finally, describe Algorithm 5.2 from Ragavan and Vaikuntanathan[11] in terms of what we wrote earlier. It is important to note that this is applied to the shifted exponent $e_i = c_i + D/2$.

Algorithm 4 Fibonacci multi-exponentiation (top-down)

Input: Initial quantum state $|e\rangle$

Output: State consisting of a superposition $\prod_{i=1}^{d} b_i^{e_i} \pmod N$

1: Compute $c_{i,j}$ in superposition according to our initial algorithm.
2: $R \leftarrow 1$
3: **for** $j = 0$ to m **do**
4: Compute $|C_j\rangle$ in the manner we described in Chapter 13, entangling $thee_i$ in a superposition.
5: Run the multiplication circuit on C_j and C_j^{-1} j times to get the next Fibonacci base
6: $R \leftarrow RC_j^{F_j}$
7: Run the multiplication circuit again j times to turn $|C_j^{F_j}\rangle\,|C_j^{-F_j}\rangle \rightarrow |C_j\rangle\,|C_j^{-1}\rangle$
8: Uncompute the intermediate nodes used to compute $|C_j\rangle$ using $|C_j^{-1}\rangle$
9: **end for**
10: R_1 is now entangled with $|e\rangle$, and holds the superposition of the possible products.
11: **return** R

We can think of C_j as a multiplication factor that is applied in each individual step, just as Regev would apply, for example, $b_1 b_3 b_4$.

We should mention that the classical postprocessing differs a bit, specifically the part we outlined in Chapter 10. With respect to complexity, given a circuit that implements something similar to (13.2) with $S = n + O(1)$ qubits and G gates, the above algorithm can be executed with $S + (A/log\phi + 6 + O(1))n$ qubits and $O(\sqrt{n}G + n^{3/2})$ gates assuming an arbitrary multiplication circuit. Assuming we adapt the classical Van Der Hoeven algorithm for multiplication[5], this is equivalent to $O(nlogn)$

qubits and $O(n^{3/2}logn)$ gates, achieving a significant improvement over Regev's $O(n^{3/2})$ qubits.

Chapter 16

Regev optimizations: Spooky pebbling

Whilst Fibonacci exponentiation does provide somewhat of a solution in a qubit-scarce environment and guarantees asymptotically better results, experiments seem to suggest that the prefactors dominate the expression for practical uses as demonstrated in [6] (which also happens to be the paper detailing the idea we will talk about in this chapter), such as factoring RSA numbers. For our final chapter, we will present an idea based on a technique called "spooky pebbling" that promises to finally transform Regev into a solution that truly leverages the quantum advantage in factoring large numbers.

First,let us describe what a pebble game is, and its relation to quantum computing:
Suppose we have a directed acyclic graph, in which each node represents an intermediate value. In every step, we can perform two operations on that graph:

1.**Pebble:** Place a pebble on any given set of nodes, provided all of their parents have pebbles. This corresponds to computing some value.
2.**Unpebble:** You can remove a pebble from a set of nodes. This corresponds to uncomputing a value, or emptying certain registers.

The goal of the game is to go from the initial no pebble state to the state

where all outputs nodes have pebbles while minimizing amount of operations (number of times you pebbled/unpebbled) and space (maximum number of pebbles currently present on the graph). As we mentioned in the previous chapter, quantum operations need to be reversible. This means that in order to correspond a quantum circuit to a pebble game, we need to add the restriction that you can only unpebble when all parent nodes of every node we are unpebbling have a pebble.

Let us now think of nodes as qubit registers.In a normal quantum circuit, qubit registers containing information need to be cleaned when they dont need to be used anymore for two reasons. Firstly, due to entanglement, it is possible they contain information that affects the final output. Secondly, qubits are expensive, and therefore reusing qubits is essential to quantum computations. Fortunately, the way quantum circuits are designed (unitary gates), every operation is reversible. This, means however, that we need to essentially run the gates in the circuit twice to "unpebble" the registers, in order to make sure the qubits are empty when we do our final measurement.

To resolve this issue, let us define a third operation, called "**Ghosting**". Given a register x, Ghosting corresponds to measuring a register in the Hadamard basis e.e. measuring after applying a Hadamard gate. A Hadamard gate on the parent disentangles the parent from the child, meaning the superposition of the parent stays intact even after the measurement. This process, though, isn't entirely clean. For a $|+\rangle$ measurement (0), everything works as normal. However, if we measure $|-\rangle$ (1), that means a relative phase of -1, or 180 degrees, has been added to the parent registers. In other words, we get the following state transition:

$$\sum |x\rangle \mapsto \sum -1^{f(x)} |x\rangle \tag{16.1}$$

for some function $f(x)$ related to the information held in the measured qubit.

The most important advantage to doing this, is that the qubit is now truly free to reuse, and the information it was carrying is now simply passed onto the results in other qubits through an already well-known process in quantum algorithms known as phase kickback. Furthermore, contrary to unpebbling, ghosting can actually be done in parallel. In order to unpebble a parent, you must first unpebble its child, making this a process that takes multiple time steps. You can, however, ghost multiple nodes at once, since the information transfers immediately through entanglement.

Now, let us assume that for our problem, we are trying to compute $H^l(x)$ for some function H. We define "blasting" as a procedure where, starting from an initial state with pebbles, we march all the way to position l, placing pebbles along the way. We will further define "unblasting" as ghosting a certain amount of qubit registers.

We will also call the total number of steps we will take in the game to reach an output node the "pebbling depth", and we will subdivide our computations into "windows" of blasting. Those windows will depend on certain "marker pebbles" we will be leaving behind in strategic positions, in order to make the blasting and unblasting process more efficient. Those markers will tell us how many qubits we should unblast every time. An important thing to note is that it is also not necessary to remove the marker every time. It could also be optimal to leave a marker intact after unblasting its children and remove it later.This lets us keep markers on certain useful values (say, perhaps, the powers of 2 in a binary representation).

In terms of quantum circuits, our objective is to minimize the pebbling depth (number of operations) given a fixed amount of pebbling space (qubit registers). This is important because, evidently, the optimal marker placement given infinite pebbling space is the binary representation of l, due to the rules of entropy. However, with our fixed number of qubits, this isn't necessarily the case, since we will not always have enough qubits to

cover every logarithmic window -in fact, in this case we would be using way more qubits than necessary.

Finally, note that most of what we described is not necessarily restricted to a line. In fact, since the Regev product necessitates computing powers of multiple bases, this approach is even more efficient, since we can parallelize our operations across an entire graph of powers, and apply markers in the most optimal positions. To find those positions, we can simply apply a variation of the A^* search algorithm in a graph consisting of all the bases and their powers, and blast/unblast multiple pebbles accordingly.

Because A^* optimizes for a given maximum space, it is impossible to run out of qubits during parallelization. Hence, we can keep blasting and unblasting according to the optimal strategy decided by A^*, and even parallelize the operations when they don't interfere with each other. This parallelization across the powers is what makes the algorithm extremely fast. Furthermore, keep in mind that the markers, besides telling us how far back we need to unblast, can also help in keeping track of the phase changes for the final gate. Finally, all of this implies that the only time we'll actually use the expensive Unpebble operation will be when uncomputing markers or the final result.

We will mostly concern ourselves here with how spooky pebbling relates to Regev. The optimal strategy for the pebbling game, including the blasting and unblasting subroutines, can be found in pages 13 and 14 as Algorithms 2.1,2.2 and 2.3 of [6], while section B in page 36 describes the logic behind applying A^* and its heuristic. Here, we will only show how pebbling and unpebbling works in terms of our computations. Finally, let us describe the application of spooky pebbling to computing a Regev product.

Algorithm 5 Pebble(i)

Input: b, N, window size w, pebble index i,quantum registers e consisting of a superposition of e_j,in and out

Output: $out = y_i$, all other registers unchanged

1: $in \leftarrow y_{i-1}$,$out \leftarrow 0$
2: **if** $i \not\equiv 1 \pmod{w+1}$ **then**
3: $\quad out \leftarrow in^2 \pmod{N}$
4: **else**
5: $\quad$ Compute $i' = (i-1)w/(w+1)$
6: $\quad p = \prod_j b_j^{2e_j(i'..i'+w)}$ where $e_j(i'..i'+w)$ denotes the integer formed by taking the bits of e_j starting from i' and ending at $i'+w$
7: $\quad out \leftarrow in \cdot p \pmod{N}$
8: $\quad Ghost(p)$
9: **end if**

Algorithm 6 Unpebble(i)

Input: b, N, window size w, pebble index i,quantum registers e consisting of a superposition of e_j,in and out

Output: $out = 0$, all other registers unchanged, all intermediate ghosts removed

1: $in \leftarrow y_{i-1}$,$out \leftarrow y_i$
2: **if** $i \not\equiv 1 \pmod{w+1}$ **then**
3: $\quad out \leftarrow out - in^2 \pmod{N}$
4: **else**
5: $\quad$ Compute $i' = (i-1)w/(w+1)$
6: $\quad p = \prod_j b_j^{2e_j(i'..i'+w)}$ where $e_j(i'..i'+w)$ denotes the integer formed by taking the bits of e_j starting from i' and ending at $i'+w$
7: $\quad out \leftarrow out - in \cdot p \pmod{N}$
8: $\quad Unghost(p)$
9: $\quad p \leftarrow p - \prod_j b_j^{e_j(i'..i'+w)}$
10: **end if**

The complexity depends on the length of the chain. According to the paper [6], this amounts to $l = 2logD$. Given $2\sqrt{d}R \leq D \leq 4\sqrt{d}R$, and $R > \sqrt{2d}$, we get $l = O(d)$. This means that $l = O(d) = O(\sqrt{n})$. The pebbling space is $2.47logl$ pebbles, so in total $O(logn)$ pebbling space.

Using these results, we get $4logD = O(\sqrt{n})$ circuit depth with multiplications, which is optimal across each run, amounts to almost half the per-run depth of optimized Shor and can be parallelized. Qubit count is bounded up by $1.3(2n + S + o(n))loglogD$ using our earlier results, which simplifies to $O(nlogn)$ qubits using the dominant $2n$ term. Lastly, the authors claim that across all independent runs, the total number of multiplications executed is approximately half an order of magnitude better than optimized Regev circuits. Results from [6], page 4 are pasted below. Note that the BKZ algorithm is preferred for lattice reduction by the paper's authors.

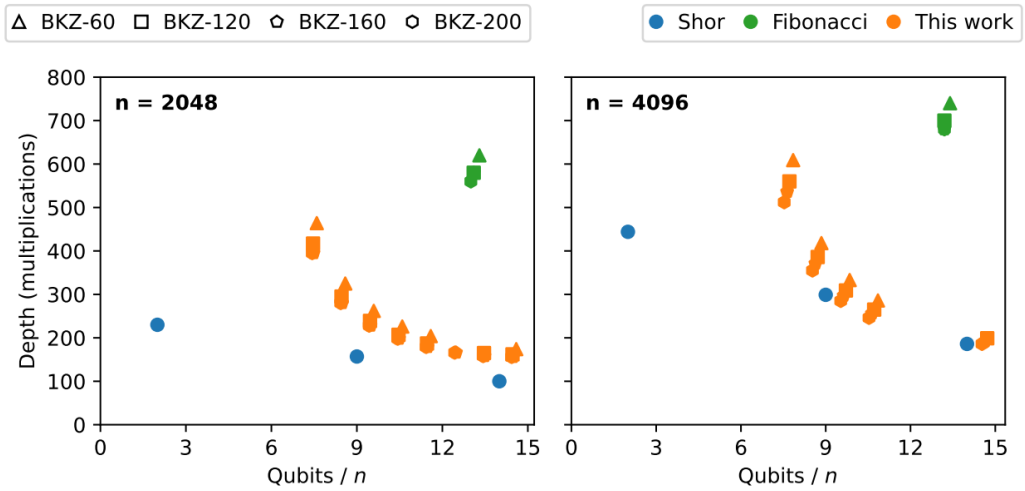

Figure 16.1: Spooky pebbling comparison.

BIBLIOGRAPHY

[1] Boris Adamczewski and Yann Bugeaud. On the complexity of algebraic numbers II. continued fractions. *Acta Mathematica*, 195:1–20, 2005.

[2] Jr. Burton S. Kaliski. Targeted fibonacci exponentiation. *IACR Cryptology ePrint Archive*, 2017:paper 260, 2017.

[3] Martin Ekera. On completely factoring any integer efficiently in a single run of an order-finding algorithm. *Quantum Information Processing*, 20(205), 2021.

[4] Craig Gidney and Martin Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021.

[5] David Harvey and Joris van der Hoeven. Integer multiplication in time $o(n \log n)$. *Annals of Mathematics*, 193(2):503–592, 2021.

[6] Grant Kahanamoku-Meyer, Seyoon Ragavan, and Katherine Van Kirk. Parallel spooky pebbling makes regev factoring more practical. *IACR Cryptology ePrint Archive*, 2024:123, 2024.

[7] Katharina Kreuzer. Np-hardness of cvp and svp. *Archive of Formal Proofs*, 2025.

[8] Youness Lamzouri, Xiannan Li, and Kannan Soundararajan. Conditional bounds for the least quadratic non-residue and related problems. *Mathematics of Computation*, 84(295):2391–2412, 2015.

[9] Yunseong Nam, Yuan Su, and Dmitri Maslov. Approximate quantum fourier transform with $o(n \log(n))$ t gates. *npj Quantum Information*, 6:26, 2020.

[10] Cedric Pilatte. Unconditional correctness of recent quantum algorithms for factoring and computing discrete logarithms. *Forum of Mathematics*, 14:e5, 2024.

[11] Seyoon Ragavan and Vinod Vaikuntanathan. Space-efficient and noise-robust quantum factoring. *IACR Cryptology ePrint Archive*, 2023:1501, 2023.

[12] Oded Regev. Quantum computation and lattice problems. *SIAM Journal on Computing*, 33(3):738–760, 2004.

[13] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):34:1–34:40, 2009.

[14] Oded Regev. An efficient quantum factoring algorithm. *Journal of the ACM*, 72(1):1–13, 2025.

[15] Rich Rines and Isaac L. Chuang. High performance quantum modular multipliers. *arXiv*, 2018.

[16] Martin Roetteler, Michael Naehrig, Krysta M. Svore, and Kristin Lauter. Quantum resource estimates for computing elliptic curve discrete logarithms. *Lecture Notes in Computer Science*, 10625:241–270, 2017.

[17] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.